**Algorithm 1** Top-Level Multi-Head Attention Accelerator Orchestration (`attention_core_top`)

**Require:** Matrices $Q, K, V \in \text{INT8}^{N \times d_k}$ per head stored in off-chip DDR4 memory;
Dyadic scaling parameters: Mantissa $M \in [0, 2^{18} - 1]$, Exponent $E \in [0, 31]$;
Softmax Exp-LUT parameters ($1024 \times 20$-bit fixed-point coefficients in DDR4);
Constants: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$, Number of heads $H = 4$;
AXI4-Stream bus width $W_{\text{bus}} \in \{64, 128\}$ bits;
Control flags: `start_pulse`, `update_lut_cfg`, `clear_buf`, `soft_reset`.
**Ensure:** Context output matrix $O \in \text{INT8}^{N \times d_k}$ per head written back to DDR4 memory;
Core status flag `DONE` asserted upon completion.

**if** `update_lut_cfg` is **true then**
    Load 1024-word Exp-LUT coefficients from DDR into True Dual-Port BRAM via AXI-Lite MMIO (Reg `0x40/0x44`)
**end if**
**for each** attention head $h \in [0, H)$ **do**
    DMA_MM2S_Transfer($Q[h], K[h], V[h]$) from DDR into on-chip Ping-Pong BRAM ($Q_{\text{MEM}}, K_{\text{MEM}}, V_{\text{MEM}}$)
    **for each** query token row $i \in [0, N)$ **do**
        $\mathbf{s}_{\text{raw}}[i, :] \leftarrow$ Stage1_QKT_GEMM($Q_{\text{MEM}}[i, :], K_{\text{MEM}}, N, d_k$) ▷ *Call Algorithm 2*
        $\mathbf{s}_{\text{scaled}}[i, :] \leftarrow$ Stage2_Dyadic_Scaler($\mathbf{s}_{\text{raw}}[i, :], M, E, N$) ▷ *Call Algorithm 3*
        $\mathbf{P}[i, :] \leftarrow$ Stage3_Hardware_IntSoftmax($\mathbf{s}_{\text{scaled}}[i, :], N$) ▷ *Call Algorithm 4*
        $\mathbf{o}_i \leftarrow$ Stage4_Context_GEMM($\mathbf{P}[i, :], V_{\text{MEM}}, N, d_k$) ▷ *Call Algorithm 5*
        Store token vector $O[i, :] \leftarrow \mathbf{o}_i$ into Egress Ping-Pong Buffer
    **end for**
    DMA_S2MM_Transfer($O$) to DDR4 via AXI4-Stream Egress ($W_{\text{bus}}$ beats, `tlast` asserted at final beat)
    Assert core `DONE` interrupt and update `STATUS` register (Reg `0x04`[1] $\leftarrow$ 1)

**Algorithm 2** Stage 1 — Generalized $QK^T$ GEMM Row Computation & Microarchitecture Mapping

**Require:** Query row vector $\mathbf{q}_i = Q[i, 0 : d_k - 1] \in \text{INT8}^{1\times d_k}$;
Key matrix $K \in \text{INT8}^{N\times d_k}$ stored in on-chip Ping-Pong BRAM ($K_{\text{MEM}}$);
Parameters: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$, Bus width $W_{\text{bus}} \in \{64, 128\}$;
Architecture Variant:

- **Type A** (Row-Stationary Vector Streaming): Baseline, Variant 1A (LUT tree), Variant 2A (DSP pack);
- **Type B** (2D Output-Stationary Systolic Array): Variant 3B ($4 \times 4$ PE grid with dual-INT8 packing).

**Ensure:** Raw Attention Score row vector $\mathbf{s}_{\text{raw}}[i, 0 : N - 1] \in \text{INT32}^{1\times N}$.

**if** Architecture == Type A (Row-Stationary Vector Streaming Engine) **then**
  Latch query row vector $\mathbf{q}_i$ into local registers ▷ *Stationary across all N key tokens*
  **for each** key token index $j \in [0, N)$ **do**
    **if** Variant == Variant 1A (Full-Parallel Pure LUT Tree, $N_{\text{pe}} = 32$) **then**
      Fetch key token vector $K[j, 0 : d_k - 1]$ from BRAM ▷ *1 beat/token on 128-bit bus, zero-padded*
      **for each** channel $c \in [0, 32)$ **parallel do**
        $\text{prod}[c] \leftarrow (c < d_k)\ ?\ \big(\mathbf{q}_i[c] \times K[j, c]\big) : 0$ ▷ *32 parallel INT8 multipliers in LUT*
        $s_{\text{raw}}[i, j] \leftarrow \textsc{Pipelined_Adder_Tree_5_Stages}\big(\text{prod}[0 : 31]\big)$ ▷ *Throughput = 1 cycle (II = 1)*
      Variant == Variant 2A (Multi-Beat DSP Packing, 8 DSP48E2 Slices)
      Initialize DSP accumulator register: $\text{acc}_{\text{dsp}} \leftarrow 0$
      $n_{\text{beats}} \leftarrow \lceil d_k/8 \rceil$ ▷ *2 beats for $d_k = 16$; 3 beats for $d_k = 20, 24$*
      **for each** beat $b \in [0, n_{\text{beats}})$ **do**
        Fetch 64-bit slice $K[j, 8b : 8b + 7]$ from BRAM
        **for each** DSP slice $p \in [0, 8)$ **parallel do**
          $\text{dsp_prod}[p] \leftarrow \mathbf{q}_i[8b + p] \times K[j, 8b + p]$ ▷ *Dual-INT8 packed multiply in DSP48E2*
          $\text{acc}_{\text{dsp}} \leftarrow \text{acc}_{\text{dsp}} + \sum_{p=0}^{7} \text{dsp_prod}[p]$ ▷ *Multi-beat serial accumulation*
          $s_{\text{raw}}[i, j] \leftarrow \textsc{Zero_Bubble_Serializer}\big(\text{acc}_{\text{dsp}}\big)$ ▷ *Bubbleless output sequencer*

      **end for**
      Architecture == Type B (2D Output-Stationary Systolic Array Tiling: Variant 3B)
      Constants: $T_{\text{row}} = 4$ (token rows), $T_{\text{col}} = 8$ (token columns via $4 \times 4$ PE grid with dual-INT8 packing)
      **for each** column block $j_{\text{blk}} \in [0, N)$ with step $T_{\text{col}}$ **do**
        $c_{\text{blk_col}} \leftarrow \min(T_{\text{col}}, N - j_{\text{blk}})$
        **for each** row block $i_{\text{blk}} \in [0, N)$ with step $T_{\text{row}}$ **do**
          $c_{\text{blk_row}} \leftarrow \min(T_{\text{row}}, N - i_{\text{blk}})$
          Initialize systolic local accumulator tile: $\mathbf{C}_{\text{local}}[0 : T_{\text{row}} - 1, 0 : T_{\text{col}} - 1] \leftarrow 0$
          **for each** dimension slice $k_0 \in [0, d_k)$ with step 1 **do**
            Load local $Q$ slice $Q[i_{\text{blk}} : i_{\text{blk}} + T_{\text{row}} - 1, k_0]$ into PE row inputs
            Load local $K$ slice $K[j_{\text{blk}} : j_{\text{blk}} + T_{\text{col}} - 1, k_0]$ into PE column inputs
            $\mathbf{C}_{\text{local}} \leftarrow \mathbf{C}_{\text{local}} + Q_{\text{local}} \times K^T_{\text{local}}$ ▷ *$4 \times 4$ PEs $\times 2$ INT8 = 32 MACs/cycle*
            Write tile $\mathbf{C}_{\text{local}}$ to Intermediate Score Reorder Buffer at offset $(i_{\text{blk}}, j_{\text{blk}})$
            Drain raw score row vector from Reorder Buffer: $\mathbf{s}_{\text{raw}}[i, 0 : N - 1] \leftarrow \text{Reorder_Buffer}[i, 0 : N - 1]$

**Algorithm 3** Stage 2 — Pipelined Dyadic Scaler Unit (`scale_unit`)

**Require:** Raw Attention Score row vector $\mathbf{s}_{\text{raw}}[i, 0 : N-1] \in \text{INT32}^{1\times N}$;
Dyadic scaling parameters: Mantissa $M \in [0, 2^{18}-1]$, Exponent $E \in [0, 31]$;
Sequence length $N = 256$.
**Ensure:** Scaled Attention Score row vector $\mathbf{s}_{\text{scaled}}[i, 0 : N-1] \in \text{INT16}^{1\times N}$ saturated to $[-32768, 32767]$.

**for each** key token index $j \in [0, N)$ **do**
  $P_{\text{mult}} \leftarrow s_{\text{raw}}[i, j] \times M$ ▷ *DSP multiplication: INT32 × INT18 → INT50*
  $S_{\text{shifted}} \leftarrow P_{\text{mult}} \ggg E$ ▷ *Arithmetic right shift by E positions*
  $s_{\text{scaled}}[i, j] \leftarrow \text{Clamp}_{\text{INT16}}(S_{\text{shifted}})$ ▷ *Saturation clamp to* $[-32768, 32767]$
  **return** $\mathbf{s}_{\text{scaled}}[i, 0 : N-1]$

**Algorithm 4** Stage 3 — Reconfigurable 3-Pass Hardware IntSoftmax Engine (`softmax_lut`)

**Require:** Scaled Attention Score row vector $\mathbf{s}_{\text{scaled}}[i, 0 : N-1] \in \text{INT16}^{1\times N}$;
True Dual-Port Exp-LUT BRAM table EXP_BRAM_LUT[0 : 1023] (1024 × 20-bit);
Sequence length $N = 256$.
**Ensure:** Normalized Attention Probability row vector $\mathbf{P}[i, 0 : N-1] \in \text{UINT16}^{1\times N}$ where $P[i, j] \in [0, 32767]$.

▷ ***Pass 1: Streaming Row Maximum Search***
$m_i \leftarrow -32768$ ▷ *Initialize to minimum INT16 value*
**for each** key token index $j \in [0, N)$ **do**
  $m_i \leftarrow \max(m_i, s_{\text{scaled}}[i, j])$ ▷ *Dynamic peak tracking comparator*
**end for**

▷ ***Pass 2: Exponentiation via BRAM LUT & Denominator Summation***
$S_i \leftarrow 0$ ▷ *Initialize 24-bit denominator accumulator*
**for each** key token index $j \in [0, N)$ **do**
  $d_{i,j} \leftarrow s_{\text{scaled}}[i, j] - m_i$ ▷ *Subtract max: guaranteed* $d_{i,j} \le 0$
  lut_addr $\leftarrow \text{Clamp}_{\text{INT}}(-1023, 0, d_{i,j}) + 1023$ ▷ *Bound check to BRAM index* $[0, 1023]$
  $e_{i,j} \leftarrow$ EXP_BRAM_LUT[lut_addr] ▷ *Tra giá tr hàm mũ nh dng UFix20_19*
  $S_i \leftarrow S_i + e_{i,j}$ ▷ *Accumulate sum* $\sum e^{s_j - m_i}$
**end for**

▷ ***Intermediate: 32-Step Restoring Integer Division***
$R_i \leftarrow$ 32_Step_Restoring_Divider$(2^{31}, S_i)$ ▷ *Reciprocal multiplier* $R_i = \lfloor 2^{31}/S_i \rfloor$ *(UFix32_31)*

▷ ***Pass 3: Probability Normalization & Saturation***
**for each** key token index $j \in [0, N)$ **do**
  $M_{\text{prob}} \leftarrow e_{i,j} \times R_i$ ▷ *Multiplier: 20-bit × 32-bit → 52-bit*
  $P[i, j] \leftarrow \text{Clamp}_{\text{UINT16}}(M_{\text{prob}} \ggg 16)$ ▷ *Normalized probability in range* $[0, 32767]$
**end for**
**return** $\mathbf{P}[i, 0 : N-1]$ =0

**Algorithm 5** Stage 4 — Context Score × Value GEMM Engine (`score_v_gemm`)

**Require:** Normalized Probability row vector $\mathbf{P}[i, 0 : N-1] \in \text{UINT16}^{1\times N}$;
Value matrix $V \in \text{INT8}^{N\times d_k}$ stored in on-chip Ping-Pong BRAM ($V_{\text{MEM}}$);
Constants: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$.
**Ensure:** Context output token vector $\mathbf{o}_i = O[i, 0 : d_k - 1] \in \text{INT8}^{1\times d_k}$.

**for each** feature channel $d \in [0, d_k)$ **parallel do**
  $\text{acc}_{\text{out}}[d] \leftarrow 0$ ▷ *Initialize 32-bit channel accumulator*
  **for each** token index $j \in [0, N)$ **do**
    $\text{acc}_{\text{out}}[d] \leftarrow \text{acc}_{\text{out}}[d] + \big(P[i, j] \times V_{\text{MEM}}[j, d]\big)$ ▷ *UINT16 × INT8 → INT24 accumulation*
    $S_{\text{proj}} \leftarrow \text{acc}_{\text{out}}[d] \ggg 15$ ▷ *Fixed-point 15-bit shift right*
    $O[i, d] \leftarrow \text{Clamp}_{\text{INT8}}\big(S_{\text{proj}}\big)$ ▷ *Saturation clamp to* $[-128, 127]$
    **return** $O[i, 0 : d_k - 1]$